Milestone 1: Text Watermarking

Ian Smaglinski (solo project) | November 1, 2026

Detector and data

So to start out I used one random 32 byte key for the whole thing and I did it in a certain order cause I split the passages into sets first then I made and saved the key after that and the reason being is so my results aren't tied to any score in any way like at all. For the coloring part I used HMAC-SHA256 and basically each word gets a color off of it where if the lowest bit of the last byte is a 1 then the word is green and if its a 0 then its red. The score is super easy to figure out you just count the green words and divide that by the total number of words and above 0.65 is a flag and exactly 0.65 is not.

Finding the words was the trickier part honestly. The code runs Unicode NFC then it lowercases everything then it applies the pattern [^\W\d _]+ and that one line does a lot like a ton cause punctuation, apostrophes, hyphens, decimal digits, underscores, zero width characters, all of that splits a word while a few other unicode numbers stay put and any leftover combining marks still break things apart. I used NFC and not NFKC or casefold just cause NFC only combines accent forms when they match and I made sure watermarking and detection both use the same rules so both sides always see the same words and repeated words count and I also track how many distinct words there are and if there are no words at all the input just errors out.

All 315 passages are unique and they come from six books with three in each set and I did that split before I looked at any scores so if a passage is from a book it stays in that book's set with no exceptions like any.

Source	Set	Unmarked	For watermarking
Pride and Prejudice	Calibration	50	2
Frankenstein	Calibration	50	2
The Adventures of Sherlock Holmes	Calibration	50	1
Alice's Adventures in Wonderland	Evaluation	50	4
A Tale of Two Cities	Evaluation	50	3
Great Expectations	Evaluation	50	3

So each set has 51 unmarked passages of 30 to 99 words and another 51 of 100 to 199 words and 48 of 200 to 400 words and then there are also 15 extra passages and those are 100 to 200 words each and I dumped the source and word count for every single passage into data/manifest.json.

To get the passages the script skipped the first 10% of each book then it pulled prose paragraphs that had at least 20 words and ended in a period or something similar and some of the excerpts are missing bits of conversation and that's just cause the short paragraphs got skipped so its not a big deal. One thing I want to flag is that before I saw any scores I already picked the first five evaluation originals for the query access forgery test and those are eva-11-51 through eva-11-54 plus eva-98-51.

Calibration

The detector scored the 150 unmarked calibration passages first and after that I ran watermarking on the other five using a fixed synonym list where the code works right to left so it finds a red word and swaps it for a green synonym and it stops as soon as any of three things happen which would be the score hits 0.70 or 30 word occurrences have changed or one pass finishes, whichever comes first. I saved every original and every edited version and even the ones that failed.

Threshold	Unmarked flagged / 150	Outputs missed / 5
0.55	4	5
0.60	1	5
0.65	0	5

Picking the threshold meant adding up the false positive rate and the missed output rate and if two were tied then the higher threshold won but the thing is all three thresholds missed all five outputs so it really just came down to false positives and thats why I went with 0.65. And once that was locked in nothing else moved like at all, the key, the word rules, the threshold, the target, the synonym list, the stopping rule, all fixed for the evaluation.

Passage	Before	After	Words changed	Seconds	Quality
cal-1342-51	0.4186	0.4535	6	0.0521	Fail
cal-1342-52	0.5030	0.5269	4	0.0484	Pass
cal-84-51	0.4201	0.4379	3	0.0481	Pass
cal-84-52	0.3777	0.3883	2	0.0596	Pass
cal-1661-51	0.5133	0.5333	3	0.0397	Pass

And none of them hit the target and none of them got flagged and all of them stopped when the pass ended. A changed word is one replaced occurrence so if I swap the same word twice thats two and the total word count never changes since every swap is one red word for one green word. The time numbers only cover the automatic editing and the file saving and I didn't spend any time hand editing passages at all.

For quality two things had to hold up and that's the meaning had to stay the same and it had to still read fine. I ran the preliminary checks on all 15 pairs using a shuffled packet that hid the scores and the decisions and meaning passed every single one but readability only passed for four calibration outputs and five evaluation outputs. The calibration one that failed had "a small softened" where it should've said "a little softened" and since I'm the only one on this no teammate looked over the outputs so I figured a reviewer who knows the project and the edits might lean one way or the other like they could spot the swaps more easily or excuse them more easily, either way its not clean.

Evaluation

Passage	Total / distinct words	Before	After	Words changed	Quality

eva-11-51	170 / 104	0.5059	0.5353	5	Fail
eva-11-52	167 / 93	0.5569	0.5868	5	Fail
eva-11-53	152 / 85	0.3421	0.3750	5	Fail
eva-11-54	168 / 102	0.4821	0.5000	3	Fail
eva-98-51	148 / 92	0.4797	0.4932	2	Fail
eva-98-52	173 / 114	0.5607	0.5607	0	Pass
eva-98-53	171 / 100	0.5029	0.5088	1	Pass
eva-1400-51	168 / 102	0.4524	0.4524	0	Pass
eva-1400-52	175 / 114	0.5143	0.5543	7	Pass
eva-1400-53	126 / 78	0.4524	0.4841	4	Pass

The word counts up there are the originals and results/evaluation.json has the counts for both the originals and the outputs and all 10 attempts are in there. And not one original or output got flagged and none of them reached 0.70 and they all stopped at the end of the pass. The editing clocked in at 0.4540 seconds total and that doesn't count setup or the quality review. A few of the Alice outputs came out a little weird like I saw "believed over" in a couple of them and one Dickens output had "inquired her the question" and the failed outputs stayed in the data even though they failed.

Measurement	Result
Unmarked false-positive rate	0/150 = 0%
Detection rate on watermarking outputs	0/10 = 0%
Usable and flagged, using preliminary checks	0/10 = 0%
Attempts reaching the target	0/10

Unmarked evaluation length	Flagged / tested
30-99 words	0/51
100-199 words	0/51
200-400 words	0/48

So bottom line the swaps just never pushed the scores high enough to trip the detector and I didn't tweak anything based on these results. The scripts got simplified after the fact but the rules and the saved outputs and the scores didn't change because of it.

Repetition and limits

I tried repeating "the" 1000 times and that gave a score of 1.0 and it got flagged and then "book" 1000 times gave 0.0 no flag. Both examples have 1000 words but each one only has a single distinct word so its really just one color assignment being repeated and not a bunch of independent pieces of evidence so I'm keeping these separate from the passage results.

With zero errors across 150 independent representative passages the rough 95% upper error bound is 3/150 = 2% but the catch is that the passages share a lot of vocabulary and style within the three evaluation books and two of those books are by the same author so I'm

honestly not even sure they're independent and ten watermarking attempts is just too small a number to make any big claims.

Lets say there are 10,000 unmarked submissions a year and the measured rate predicts 10,000 x 0/150 = 0 false flags and if I used the 2% bound instead I'd get about 200 assuming its stuff holds and I wouldn't bet on either number to be honest. These are old books so they probably don't read anything like a student's writing and the zero false positives need to be weighed against the zero detected watermarking outputs cause a detector that flags nothing would also show zero so thats not proof of much on its own.

Threat model

A removal is when you take a flagged usable watermarked passage and make it stop getting flagged and the meaning and readability have to hold and a forgery is when you make unmarked prose get flagged and you didn't use the watermarking service to do it. In every case the algorithm and the word rules are known and query access hands you detector responses but not the key or any internal state and key access also gives you the key but still no extra admin powers. Everything in the table below is a prediction and none of it is a measured result.

Goal	No access	Query access	Key access
Removal	Rewrite the passage with no feedback: 0 queries and about 15 minutes. This could lower the score but you have no idea if it worked and the meaning might drift. A high starting score plus a careful quality check could stop a usable removal.	Try synonyms and keep the changes that lower the returned score: up to 100 queries and 15 minutes. Binary replies give you less to go on and replacements can hurt readability. Query limits and a high starting score could put an end to it.	Find red synonyms locally: 0 queries to learn colors maybe 1 final query and up to 30 edits. Meaning and grammar can take a hit. Protecting the key blocks this access and rotation helps future documents but it doesn't save results made under the old key.
Forgery	Rewrite without knowing the colors: 0 queries and about 15 minutes. There's no reliable way to bump the score and a lucky flag is possible though. Editing can hurt quality and a low false positive rate makes the lucky shot less likely and separate origin evidence can rule out a false attribution.	Query repeated words to learn their colors and then use green synonyms: 1 query per word learned plus passage checks. Suggested limits are 100 preparation queries then 100 queries and 15 minutes per passage. Quality can limit success and rate limits slow down the learning and binary replies still give away repeated word colors so hiding scores is not enough on its own.	Use the key to find green synonyms locally: 0 learning queries maybe 1 final query and up to 30 edits. Quality can still fail and restricting key access prevents this access and key rotation limits future forgery and a separate signature can check source claims.

Design questions

Key distribution. I think a trusted operator should hold the key for both watermarking and detection and the reason is simple which is anyone with the key for local detection can forge with it too so the other users shouldn't ever get the key and they just submit authorized detector queries. The operator can see the submitted text while its being checked and an independent reviewer could verify the exact text rules threshold and counts by watching a rerun and if you hand them the key they could check it locally but then they could also forge and a flag on its own doesn't prove authorship or cheating.

Context dependence. If the colors depended on the words before them then a color you learned in one spot might not work elsewhere and if you change one word the colors of later words could shift too and that'd force attacks to check more for each context, rewriting and key disclosure would still be issues though.

Digital signatures. A signature verifies that the exact bytes of a document were signed by a key so the moment you edit it the signature breaks unless the new version gets signed too and a watermark might survive some edits but it can also give you false results. Both kinds of keys can be used to forge if someone steals them and rotation helps going forward and dated records can help check older signatures but at the end of the day neither one proves who actually came up with the ideas.

Required reading. Kirchenbauer et al [1] favor green tokens while the model is generating using the earlier tokens to build the green list and a statistical test for detection and this project is a different beast cause it changes words that are already there using synonyms and it fixes each word's color with HMAC and it just checks the green fraction so the results and error guarantees from that paper don't carry over to this simpler scheme.

References

[1] Kirchenbauer et al, A Watermark for Large Language Models, ICML 2023, introduction and scheme description: https://arxiv.org/abs/2301.10226.

Book sources are linked in the source table and saved in data/manifest.json and the downloaded books keep their Gutenberg notices and are public domain in the United States. Project requirements come from project-description-2026-fall.md Sections 2 and 4.